

Projeto e Análise de Algoritmos

Tabelas de dispersão

Thiago Pinheiro de Araújo

EMAp/FGV

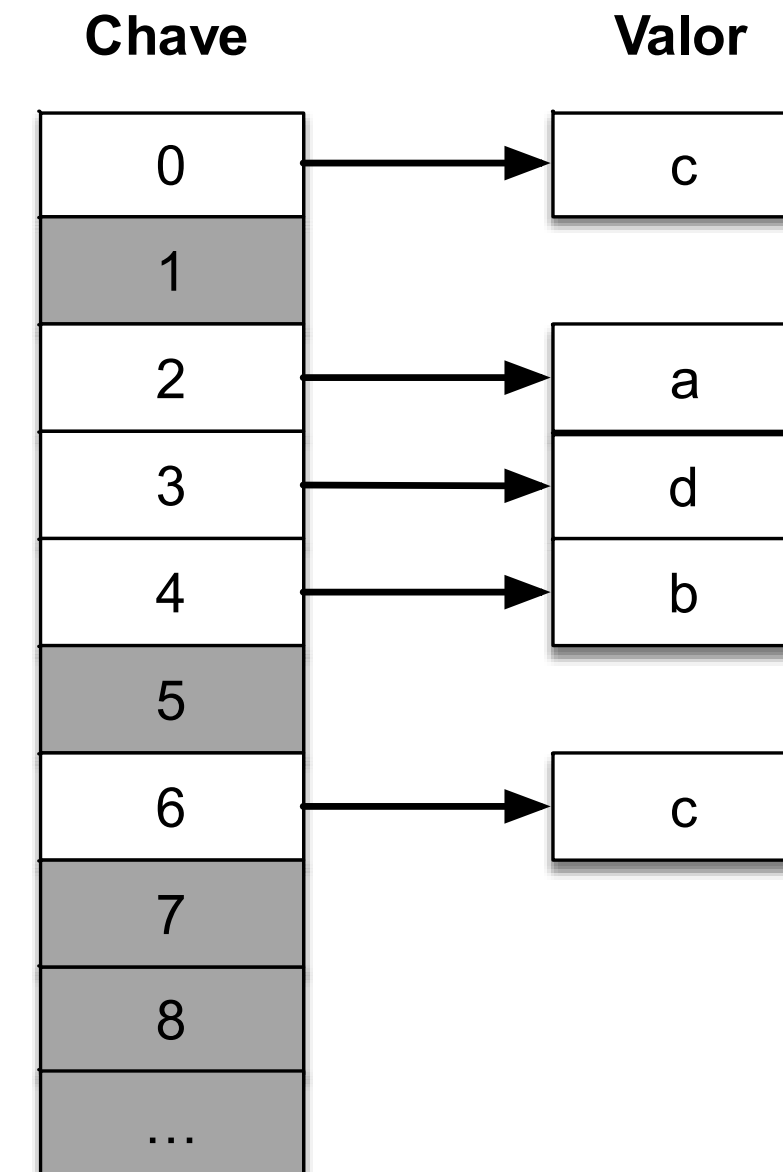
2025.2

Apresentar a estrutura de dados **Hash Table** e seus algoritmos de para inserir, buscar e remover elementos; discutindo abordagens para resolver colisões, complexidade e exemplos de implementação.

Tabelas de dispersão

Definição

- A **tabela de dispersão** (ou tabela hash) é uma estrutura de dados utilizada para armazenar e pesquisar tuplas $\langle chave, valor \rangle$ de forma eficiente.
- A tabela hash costuma ser acompanhada de algoritmos para:
 - **Inserir** um valor associado à uma chave.
 - **Buscar** o valor associado à uma chave.
 - **Remover** o valor associado à uma chave.
- As chaves contidas em uma tabela são únicas.



Tabelas de dispersão

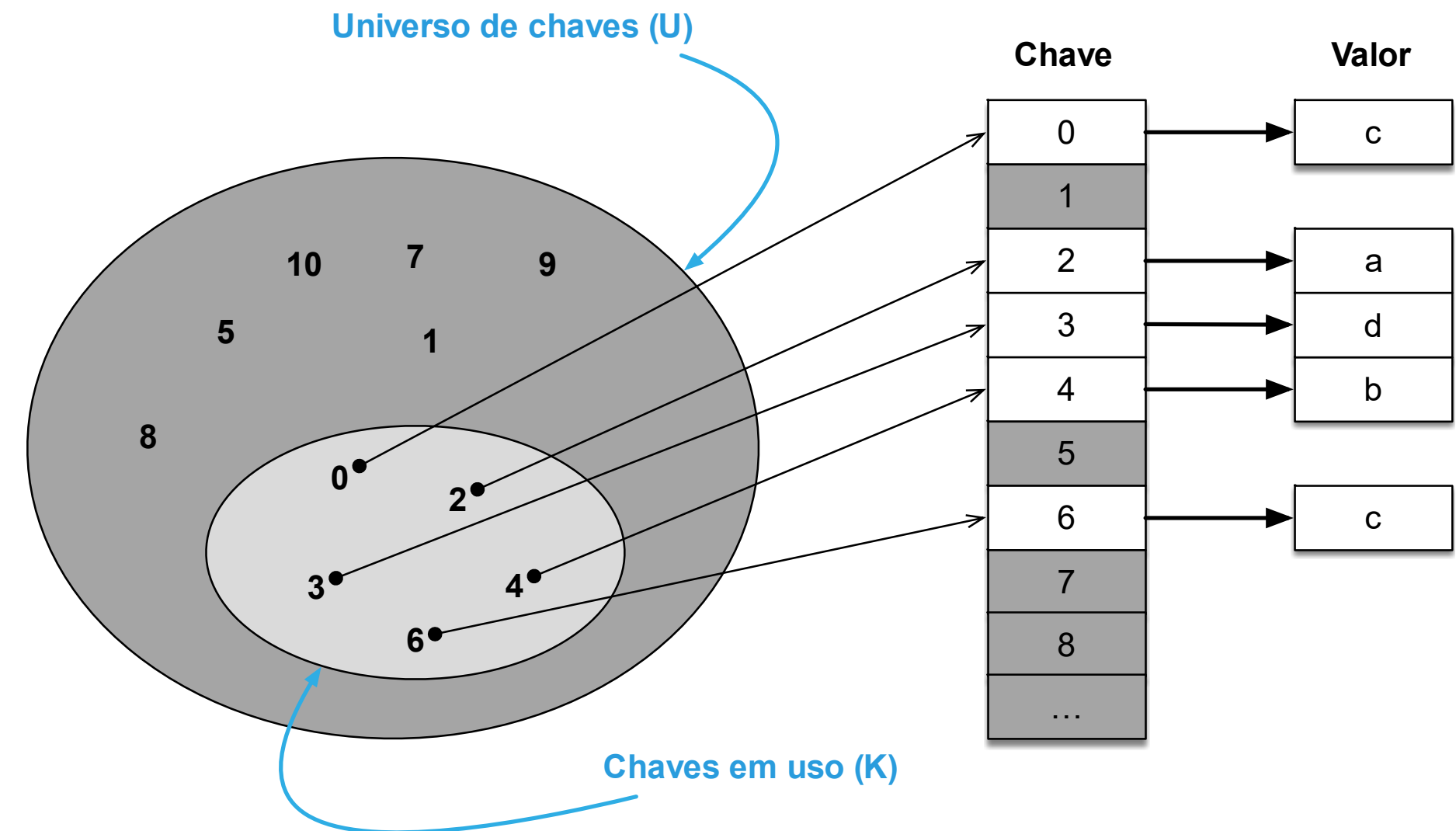
Definição

- O número de elementos existente em uma tabela hash é definido por n .

- O conjunto de chaves possíveis é chamado de Universo de chaves (U).

- O conjunto de chaves utilizadas é definido por K .

- Almeja-se executar operações sobre a tabela em $\theta(1)$.



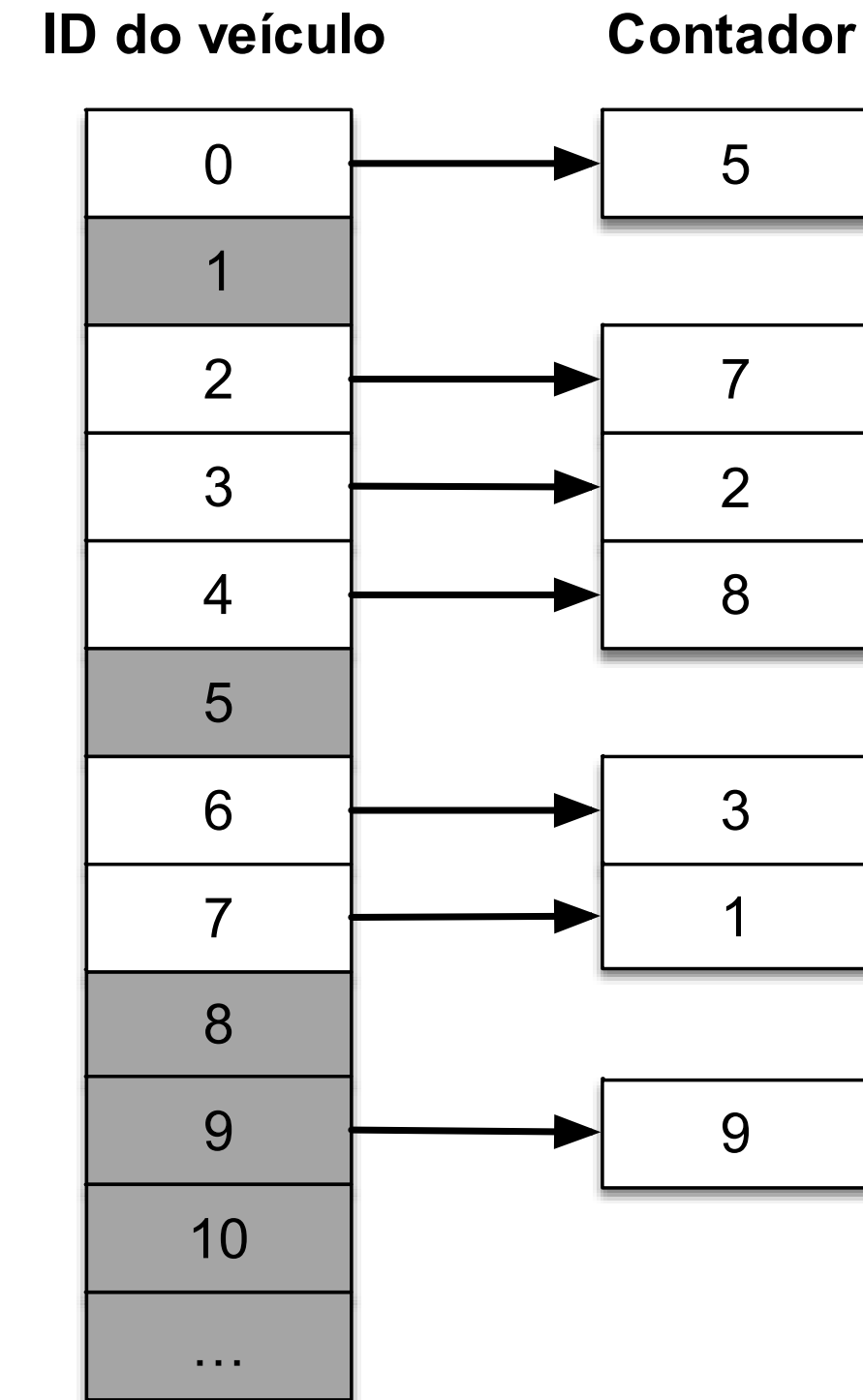
Problema motivacional

- Vamos utilizar um **problema de contagem** como exemplo para motivar a busca pela solução fornecida pela tabela hash.
- Considere um programa que recebe eventos emitidos por veículos ao entrar em uma determinada região
 - Cada evento é composto por um inteiro representando o ID do veículo.
- O programa deve contar o número de vezes que cada veículo entrou na região.
- Ocasionalmente o programa recebe uma requisição para exibir o número de ocorrências de um dado veículo.

Tabelas de dispersão

Problema motivacional

- Mandatório: a contagem deve ser incremental, sem qualquer estratégia de cache
 - Uma requisição para exibir o resultado parcial da contagem deverá contemplar todos os eventos recebidos até o momento.
- Exemplo de entrada:
`2, 6, 4, 2, 0, 4, 4, 2, 3, 7, 0, 4, 0, 2, 2, 4, 0, 3, 4, 6, 0, 2, 2, 4, 6, 4`
- Exemplo de saída (figura ao lado).



Tabelas de dispersão

Solução de busca (sem usar uma tabela hash)

- **Tentativa 1: Endereçamento direto**

- Aloca-se um vetor com o tamanho do universo U:

```
int table[U];  
for (int i=0; i < U; i++) {  
    table[i] = 0;  
}
```

- Ao processar cada evento incrementa-se a posição no vetor:

```
void add(int key) {  
    table[key]++;  
}
```

- Lê-se a contagem acessando a posição do vetor diretamente:

```
int search(int key) {  
    return table[key];  
}
```

Tabelas de dispersão

Solução de busca (sem usar uma tabela hash)

- **Tentativa 1:** Endereçamento direto.
- Análise da complexidade:
 - Inserção e busca em $\Theta(1)$.
- Ambas as operações executam em tempo constante independente do número de elementos registrados na tabela.
 - No entanto, a complexidade de espaço é $\Theta(MAX(U))$.
- Conclusão: esse algoritmo somente é eficiente quando o universo apresenta um conjunto pequeno de chaves.

Tabelas de dispersão

Solução de busca (sem usar uma tabela hash)

■ Tentativa 2: Lista encadeada

- Armazena-se os eventos em uma lista encadeada

```
typedef struct LLNode CountNode;  
struct LLNode {  
    int id;  
    int count;  
    CountNode * next;  
};
```

- Os nós são criados à medida que novos IDs são identificados.

```
void add(int key) {  
    CountNode * node = m_firstNode;  
    while (node != nullptr && node->id != key) {  
        node = node->next;  
    }  
    if (node != nullptr) {  
        node->count += 1;  
    } else {  
        CountNode * newNode = new CountNode;  
        newNode->id = key;  
        newNode->count = 1;  
        newNode->next = m_firstNode;  
        m_firstNode = newNode;  
    }  
}
```

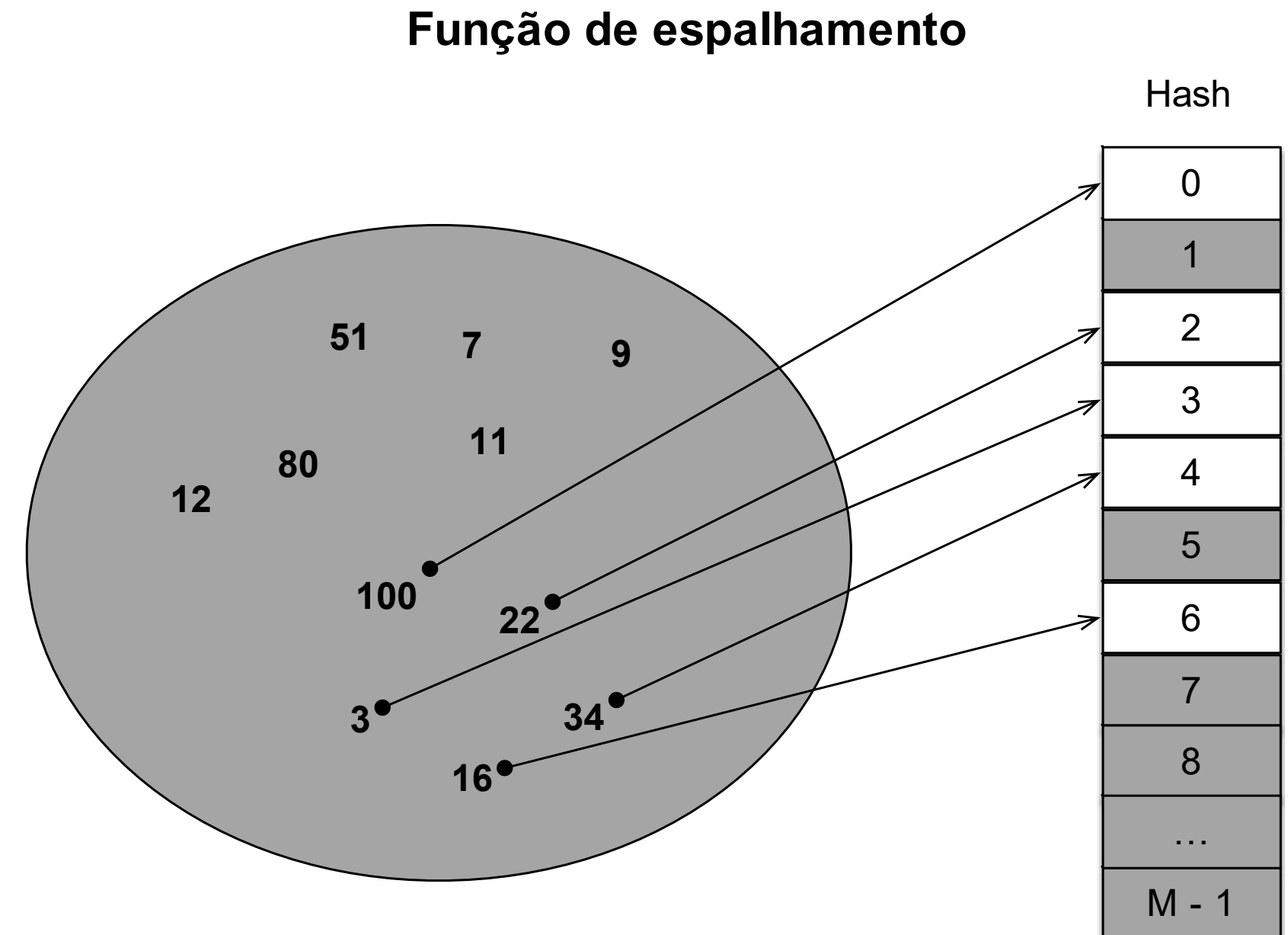
```
int search(int key) {  
    CountNode * node = m_firstNode;  
    while (node != nullptr && node->id != key) {  
        node = node->next;  
    }  
    return node != nullptr ? node->count : 0;  
}
```

- **Tentativa 2:** Lista encadeada.
- O espaço necessário para armazenar possui exatamente o número de chaves em uso.
- O tempo de inserção e busca é proporcional ao número de IDs distintos inseridos
 - Portanto, a complexidade no pior caso é $\Theta(n)$.
- Conclusão: embora o problema de espaço tenha sido resolvido (comparando com a primeira tentativa), a complexidade no pior caso é proporcional à busca linear.

Tabelas de dispersão

Definição

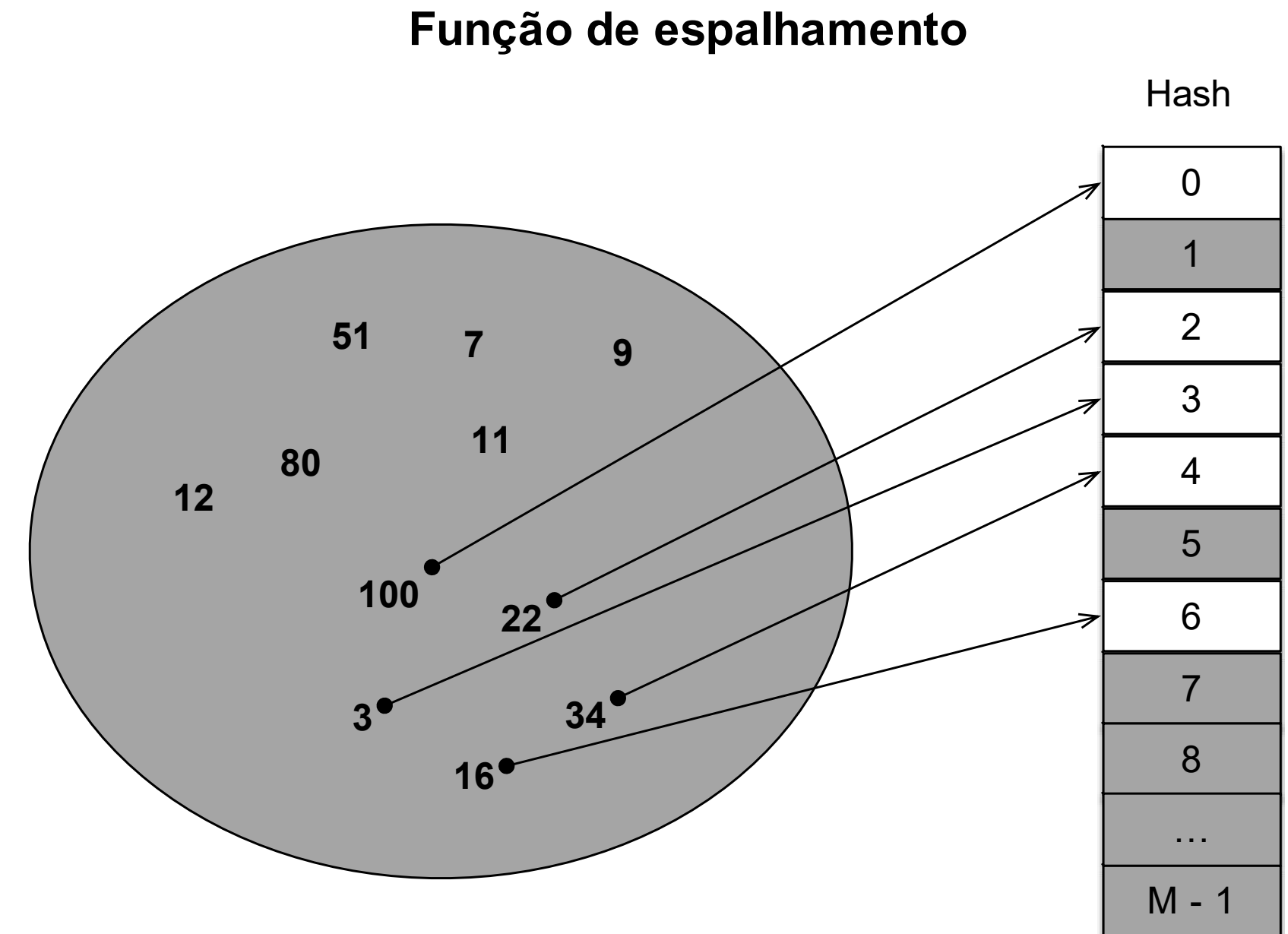
- A **tabela hash** é uma estrutura de dados baseada em um vetor de M posições acessado através de endereçamento direto
 - Por ora, mantenha a estrutura do elemento do vetor em aberto.
- Uma função de espalhamento (*hashing*) mapeia uma chave em um índice $[0, M - 1]$ do vetor
 - O resultado dessa função é comumente chamado de hash.



Tabelas de dispersão

Definição

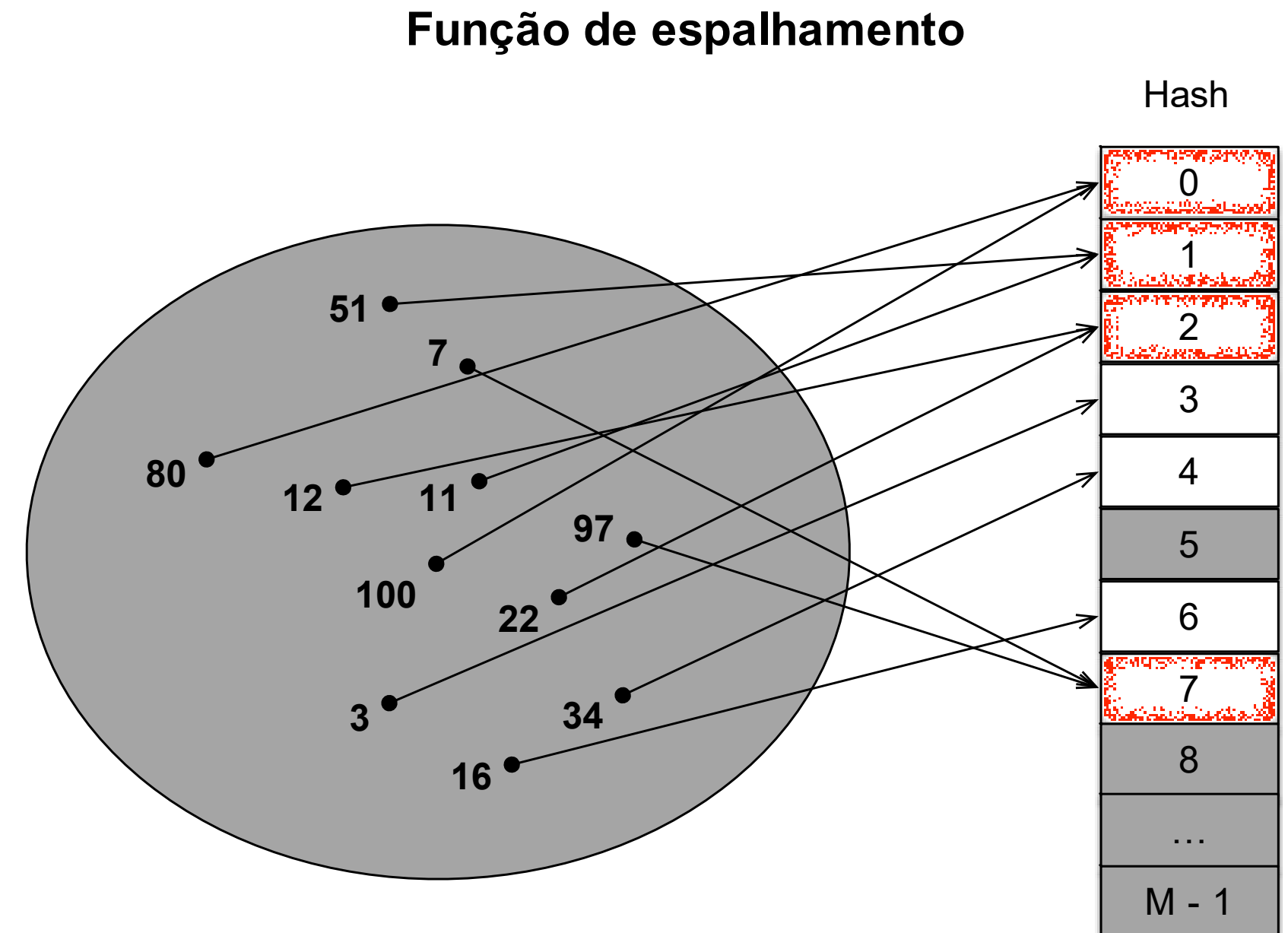
- O objetivo da função de espalhamento é reduzir o intervalo de índices de forma que M seja muito menor que o tamanho do universo U .
- Exemplo de uma função de espalhamento:
 - $hash(key) = key \% M$
- Considere também que a chave não precisa ser um inteiro e pode assumir qualquer tipo.



Tabelas de dispersão

Definição

- Quando a função de espalhamento produz o mesmo hash para chaves diferentes ocorre uma colisão
 - Existem diferentes abordagens para resolver a colisão.
- A função de espalhamento é considerada boa quando minimiza o número de colisões
 - No entanto, observe que são inevitáveis!

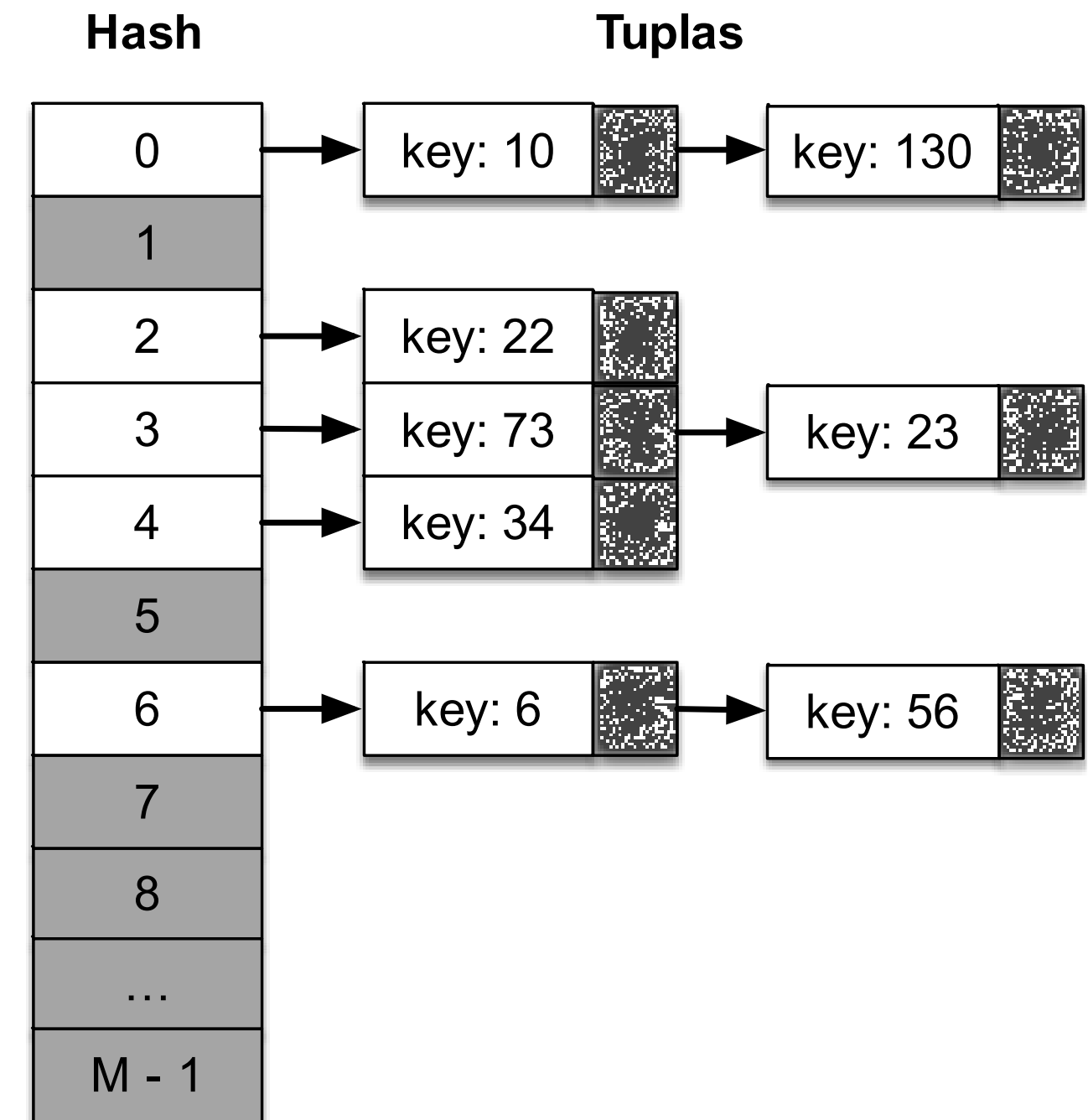


Tabelas de dispersão

Tabela hash com encadeamento

- **Abordagem 1:** Tabela hash com encadeamento
- O problema de colisão é solucionado armazenando os elementos com o mesmo hash em uma lista encadeada.
- Estrutura de um nó da lista:

```
typedef struct HashTableNode HTNode;  
struct HashTableNode {  
    unsigned key;  
    int value;  
    HTNode * next;  
    HTNode * previous;  
};
```



Tabelas de dispersão

Tabela hash com encadeamento

- Exemplo de implementação:

```
class HashTable {
public:
    HashTable(int size)
        : m_table(nullptr)
        , m_size(size) {
        m_table = new HTNode*[size];
        for (int i=0; i < m_size; i++) { m_table[i] = nullptr; }
    }
    ~HashTable() {
        for (int i=0; i < m_size; i++) {
            HTNode * node = m_table[i];
            while (node != nullptr) {
                HTNode * nextNode = node->next;
                delete node;
                node = nextNode;
            }
        }
        delete[] m_table;
    }

    ...

private:
    unsigned hash(unsigned key) const { return key % m_size; }

    HTNode ** m_table;
    int m_size;
};
```

Tabelas de dispersão

Tabela hash com encadeamento

■ Exemplo de implementação:

```
void insert_or_update(unsigned key, int value) {
    unsigned h = hash(key);
    HTNode * node = m_table[h];
    while (node != nullptr && node->key != key) {
        node = node->next;
    }
    if (node == nullptr) {
        node = new HTNode;
        node->key = key;
        node->next = m_table[h];
        node->previous = nullptr;
        HTNode * firstNode = m_table[h];
        if (firstNode != nullptr) {
            firstNode->previous = node;
        }
        m_table[h] = node;
    }
    node->value = value;
}
```

```
HTNode * search(unsigned key) {
    unsigned h = hash(key);
    HTNode * node = m_table[h];
    while (node != nullptr && node->key != key) {
        node = node->next;
    }
    return node;
}
```

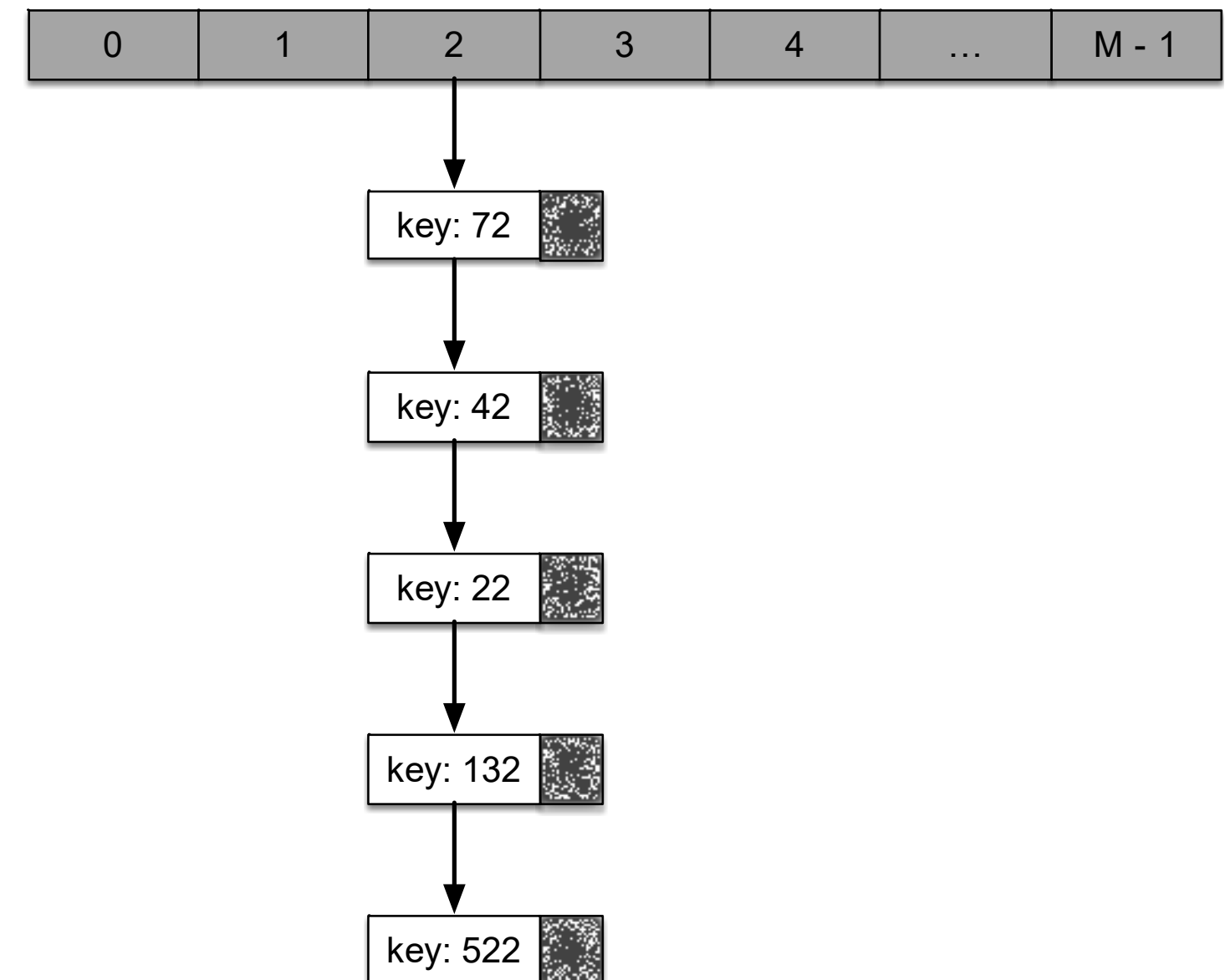

- Exemplo de implementação:

```
bool remove(unsigned key) {
    unsigned h = hash(key);
    HTNode * node = m_table[h];
    while (node != nullptr && node->key != key) {
        node = node->next;
    }
    if (node == nullptr) {
        return false;
    }
    HTNode * nextNode = node->next;
    if (nextNode != nullptr) {
        nextNode->previous = node->previous;
    }
    HTNode * previousNode = node->previous;
    if (previousNode != nullptr) {
        node->previous->next = node->next;
    } else {
        m_table[h] = node->next;
    }
    delete node;
    return true;
}
```

Tabelas de dispersão

Análise da complexidade

- Qual é a complexidade de cada uma das operações considerando o **pior caso**?
- O pior caso é atingido quando todas as chaves são mapeadas em uma única posição.
- Tempo de cada operação:
 - Inserção/atualização - $\Theta(n)$
 - Busca - $\Theta(n)$
 - Remoção - $\Theta(n)$



- A função de espalhamento ideal atende ao **hash uniforme simples**
 - Cada chave possui a mesma probabilidade de ser mapeada em qualquer índice $[0, M)$.
 - Essa é uma propriedade desejada para uma função de espalhamento a ser utilizada em uma tabela hash.
- Infelizmente esse resultado depende dos elementos a serem inseridos
 - Não sabemos à priori a distribuição das chaves ou mesmo a ordem em que serão inseridas.
- Heurísticas podem ser utilizadas para determinar uma função de espalhamento com bom desempenho.

- Alguns métodos mais comuns:
 - Simples
 - Se a chave for um número real entre $[0, 1)$
 - $hash(key) = \lfloor key \cdot M \rfloor$
 - Método da divisão
 - Se a chave for um número inteiro
 - $hash(key) = key \% M$
 - Costuma-se definir M como um número primo.
 - Método da multiplicação
 - $hash(key) = \lfloor (key \cdot A \% 1)M \rfloor$
 - A é uma constante no intervalo $0 < A < 1$.

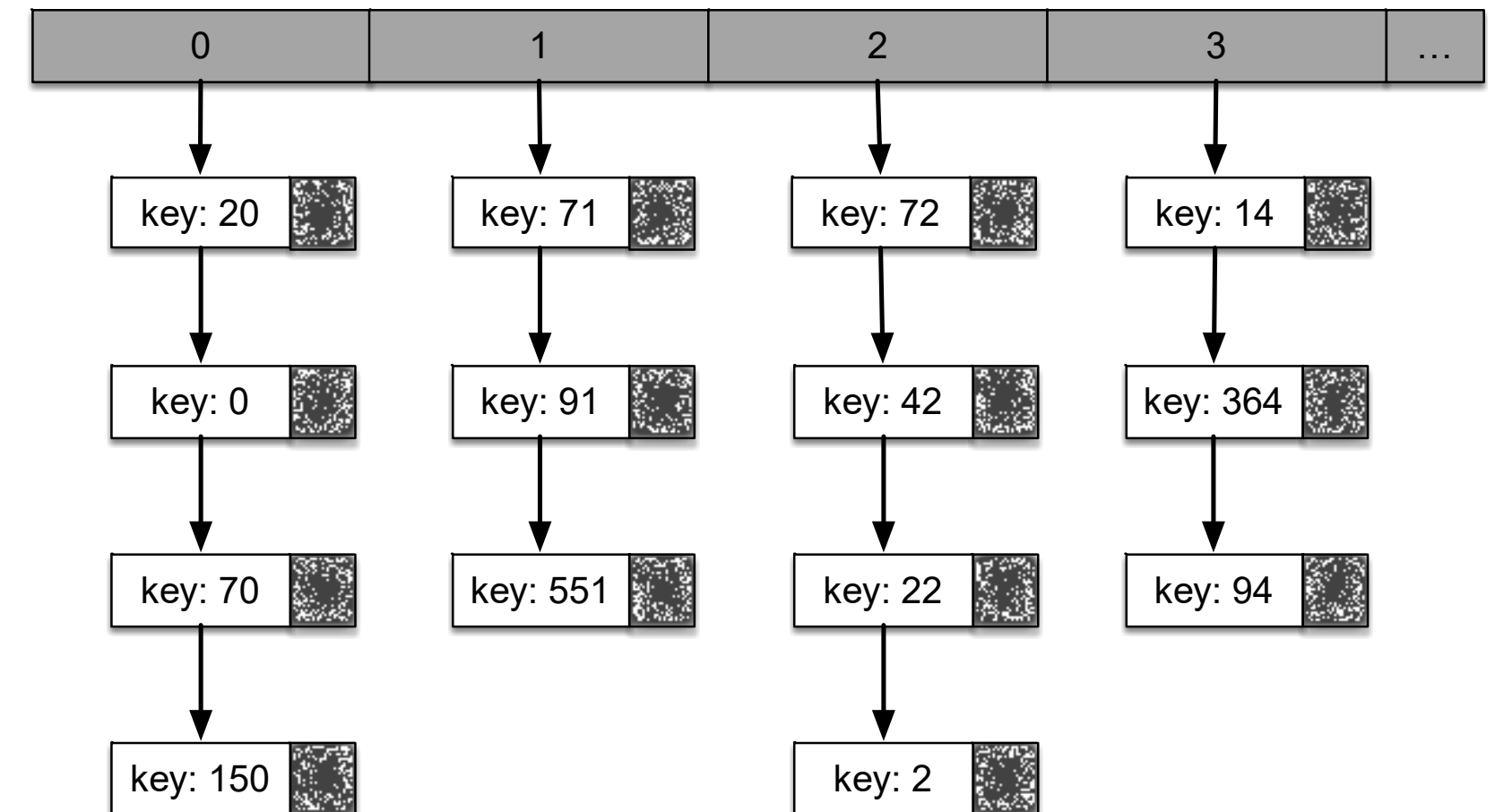
- Observe que a chave pode assumir qualquer tipo suportado pela linguagem
 - Ex: `countries["BR"]`
- A função de espalhamento é responsável por gerar um índice numérico com base no tipo de entrada
 - Uma possível função de espalhamento para strings:

```
int hashStr(const char * value, int size) {  
    unsigned hash = 0;  
    for (int i=0; value[i] != '\0'; i++) {  
        hash = (hash * 256 + value[i]) % size;  
    }  
    return hash;  
}
```


Tabelas de dispersão

Análise da complexidade

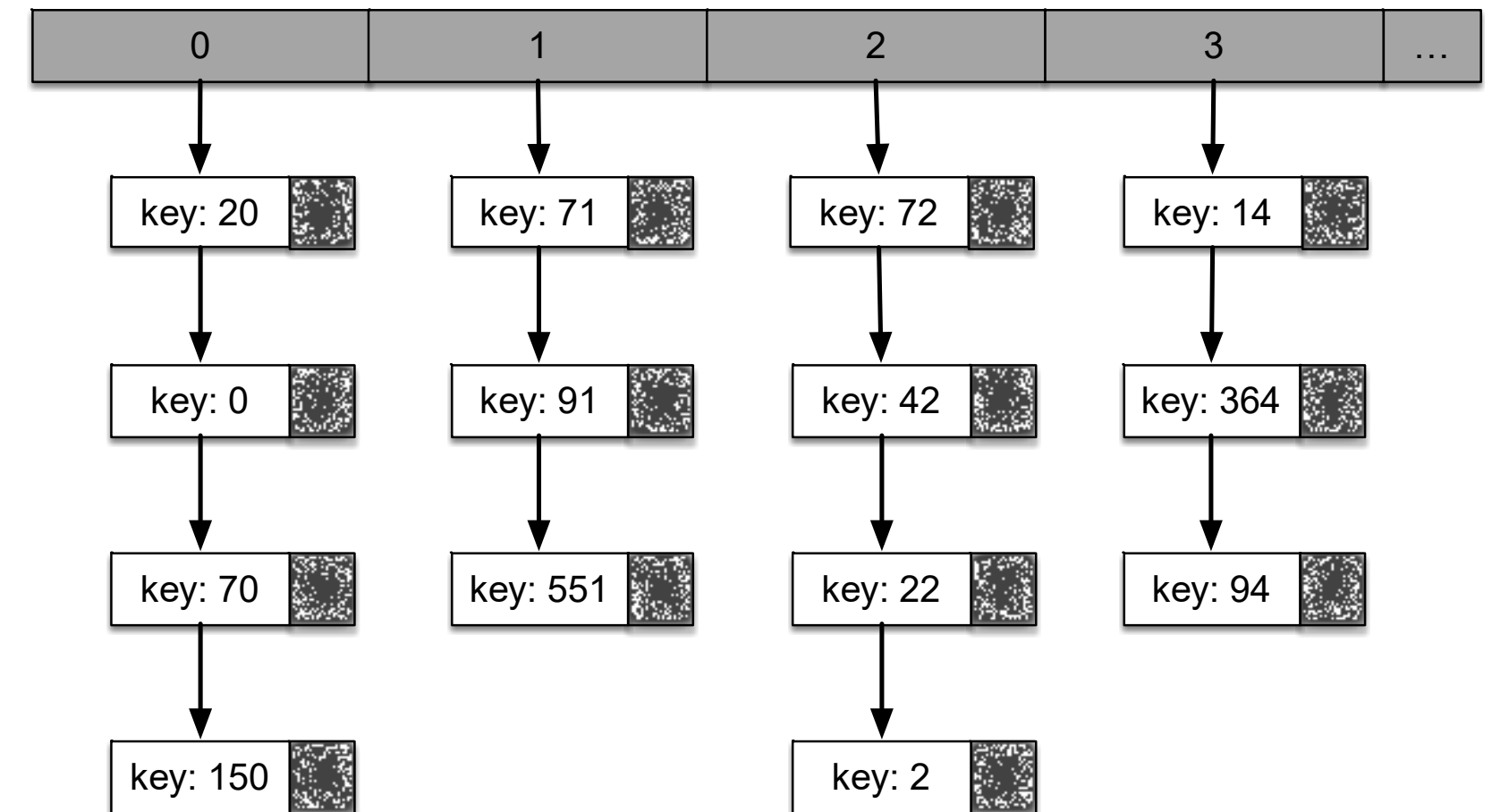
- Qual é a complexidade de cada uma das operações considerando o caso médio?
- A complexidade da função de espalhamento é constante: $\Theta(1)$
- Considerando a hipótese de hash uniforme simples podemos assumir que cada lista terá aproximadamente o mesmo tamanho.



Tabelas de dispersão

Análise da complexidade

- Qual é a complexidade de cada uma das operações considerando o caso médio?
- Podemos determinar a função de complexidade como:
 - $$T(n) = \frac{1}{n} \sum_i^n (1 + \sum_{j=i+1}^n \frac{1}{M}) = \Theta(1 + \frac{n}{M})$$
- Se M for proporcional à n podemos definir a complexidade como $\Theta(1)$
 - Porém como garantir essa característica?



- A medida que inserimos novos elementos na tabela o seu desempenho degrada com o tempo.
- Essa característica pode ser avaliada calculando o fator de carga $\alpha = n/M$
 - Esse também é o número médio de elementos armazenados em cada posição em uma tabela quando os elementos estão uniformemente distribuídos.
- Podemos avaliar se a tabela está em um estado ineficiente (carga alta) calculando esse fator a cada operação de inserção
 - Definindo um limite para o fator de carga podemos decidir se a tabela deve ser redimensionada.
 - Essa avaliação é realizada em $\Theta(1)$.

- A operação de redimensionamento deverá aumentar o tamanho de M para M' .
- Essa operação invalida o mapeamento de chaves corrente visto que os hashes foram originalmente calculados utilizando M .
- Portanto, é necessário executar uma função de re-espalhamento para voltar à um estado consistente da tabela
 - Isso pode ser realizado re-inserindo os elementos na tabela.
- A complexidade total dessa operação é $\Theta(n)$
 - Se a operação de resize & rehash tem complexidade $\Theta(n)$, como manter $\Theta(1)$ para as demais operações?

- A **análise amortizada** consiste em avaliar a complexidade com base em uma sequência de operações.
- A sequência na tabela de dispersão consiste em:
 - n operações de inserção com custo individual $\Theta(1)$.
 - k operações para redimensionamento com custo total $\sum_{i=1}^{\log(n)} 2^i = \Theta(n)$
 - Considerando que $M' = 2M$.
- Portanto, a complexidade pode ser definida por:

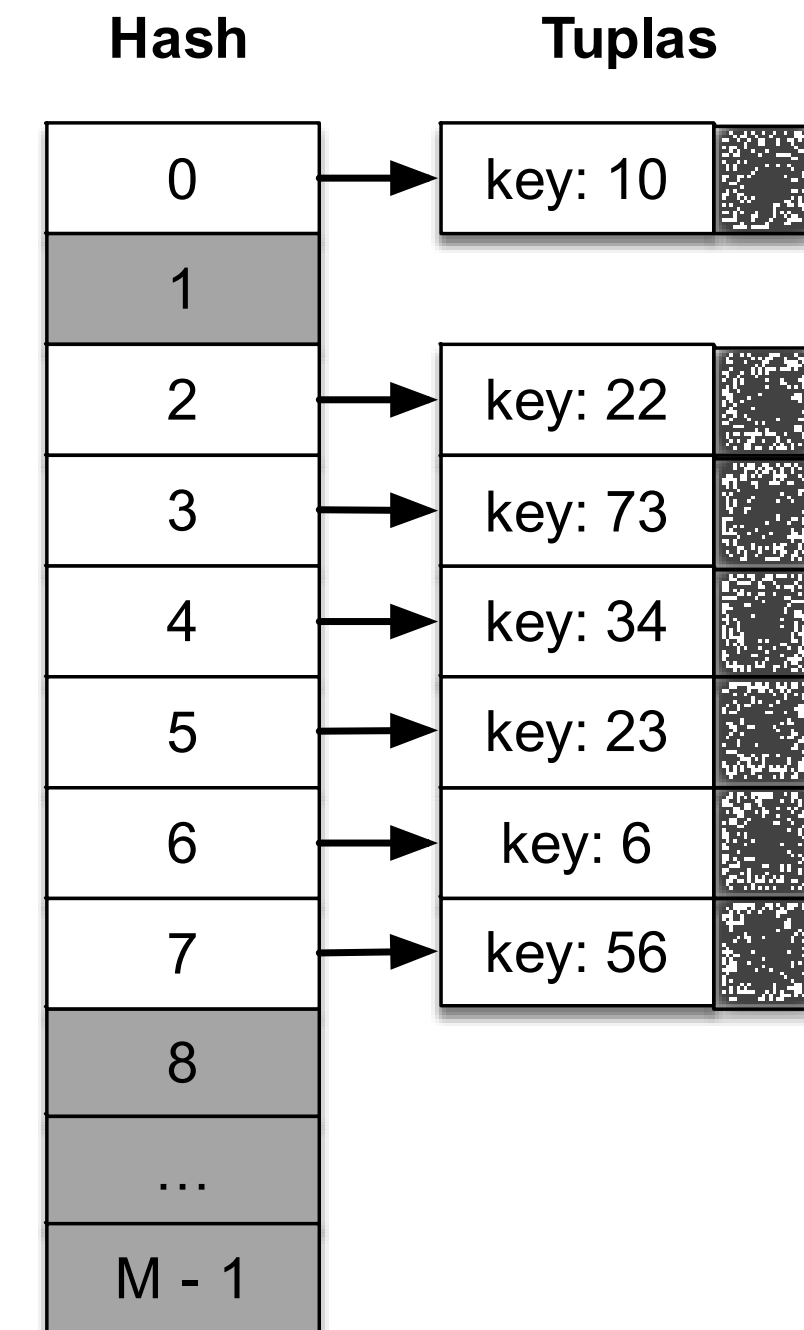
$$\frac{n \cdot \Theta(1) + \Theta(n)}{n} = \Theta(1)$$

Tabelas de dispersão

Tabela hash com endereçamento aberto

- **Abordagem 2:** Tabela hash com endereçamento aberto
- O problema de colisão é solucionado armazenando os elementos na primeira posição vazia a partir do índice definido pelo hash.
- Estrutura de um nó da lista:

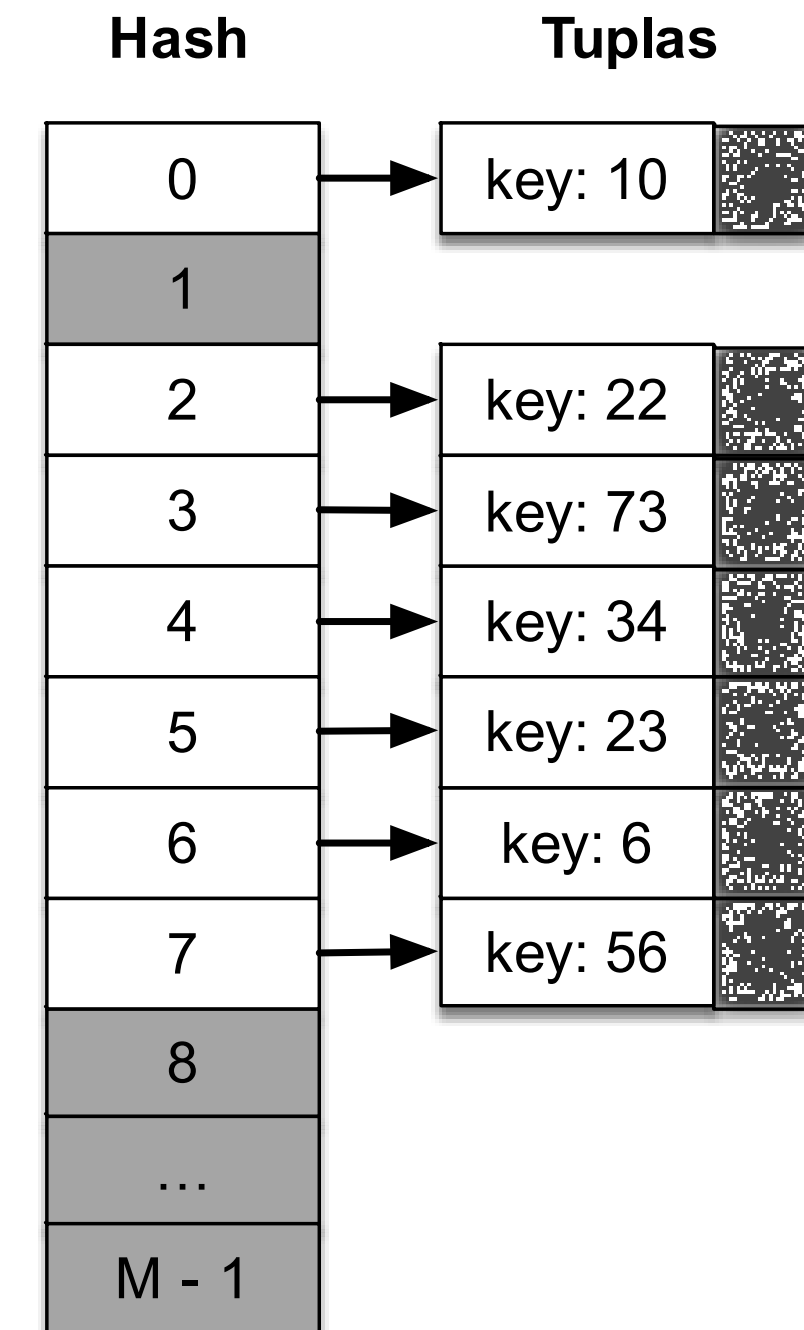
```
typedef struct DirectAddressHashTableNode DANode;  
struct DirectAddressHashTableNode {  
    int key;  
    int value;  
};
```



Tabelas de dispersão

Tabela hash com endereçamento aberto

- Ao **buscar** (ou sondar) um elemento com a chave *key*:
 - Se a posição $table[hash(key)]$ estiver
 - **vazia** – podemos garantir que a chave *key* não está presente na tabela.
 - **ocupada** – precisamos verificar se $table[hash(key)].key = key$
 - Enquanto a posição estiver ocupada e a sua chave não for a buscada deve-se avaliar a próxima posição até:
 - Encontrar a chave.
 - Encontrar uma posição vazia.
 - Voltar ao início da lista.



Tabelas de dispersão

Tabela hash com endereçamento aberto

- Exemplo de implementação da classe:

```
class DirectAddressHashTable {
public:
    DirectAddressHashTable(int size)
        : m_table(nullptr)
        , m_size(size) {
        m_table = new DNode[size];
        for (int i=0; i < m_size; i++) {
            m_table[i].key = -1;
            m_table[i].value = 0;
        }
    }
    ~DirectAddressHashTable() { delete[] m_table; }

    ...

private:
    unsigned hash(int key) const { return key % m_size; }

    DNode * m_table;
    int m_size;
};
```


Tabelas de dispersão

Tabela hash com endereçamento aberto

- Exemplo de implementação do algoritmo de inserção:

```
bool insert_or_update(int key, int value) {
    unsigned h = hash(key);
    DANode * node = nullptr;
    int count = 0;
    for (; count < m_size; count++) {
        node = &m_table[h];
        if (node->key == -1 || node->key == key) {
            break;
        }
        h = (h + 1) % m_size;
    }
    if (count >= m_size) {
        return false; // Table is full
    }
    if (node->key == -1) {
        node->key = key;
    }
    node->value = value;
    return true;
}
```

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Exemplo de implementação do algoritmo de busca:

```
DANode * search(int key) {
    unsigned h = hash(key);
    DANode * node = nullptr;
    int count = 0;
    for (; count < m_size; count++) {
        node = &m_table[h];
        if (node->key == -1 || node->key == key) {
            break;
        }
        h = (h + 1) % m_size;
    }
    return count >= m_size || node->key == -1 ? nullptr : node;
}
```

** Não é a versão final.*

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Exemplo de implementação do algoritmo de remoção:

```
bool remove(int key) {  
    DNode * node = search(key);  
    if (node == nullptr) {  
        return false;  
    }  
    node->key = -1;  
    node->value = 0;  
    return true;  
}
```

** Não é a versão final.*

Tabelas de dispersão

Tabela hash com endereçamento aberto

- A **remoção** em uma tabela hash com endereçamento aberto apresenta um problema:
 - Ao remover uma chave *key* de uma posição *h*, partindo de uma posição h_0 , tornamos impossível encontrar qualquer chave presente em uma posição $h' > h$.

Antes

M=10

hash	0	1	2	3	4	5	6	7	8	9
chave	60	21	51	131	33	91	76	61	-1	99

Depois

hash	0	1	2	3	4	5	6	7	8	9
chave	60	21	51	-1	33	91	76	61	-1	99

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Uma possível solução consiste em marcar o nó removido de forma que a busca não o considere vazio.
- Podemos criar uma *flag* para representar que o nó será reciclado.

```
typedef struct DirectAddressHashTableNode DANode;  
struct DirectAddressHashTableNode {  
    int key;  
    int value;  
    bool recycled;  
};
```

- E inicializá-la com o valor `false` no construtor:

```
m_table[i].recycled = false;
```

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Em seguida precisamos adaptar os algoritmos de busca e remoção:

```
DANode * search(int key) {
    unsigned h = hash(key);
    DANode * node = nullptr;
    int count = 0;
    for (; count < m_size; count++) {
        node = &m_table[h];
        if ((node->key == -1 && !node->recycled)
            || node->key == key) {
            break;
        }
        h = (h + 1) % m_size;
    }
    return count >= m_size
        || node->key == -1 ? nullptr : node;
}
```

```
bool remove(int key) {
    DANode * node = search(key);
    if (node == nullptr) {
        return false;
    }
    node->key = -1;
    node->value = 0;
    node->recycled = true;
    return true;
}
```

- O **fator de carga** da abordagem de endereçamento aberto é definido da mesma forma: $\alpha = n/M$
 - No entanto observe que nesse caso teremos sempre $\alpha \leq 1$ visto que M é o número máximo de elementos no vetor.
- A busca por uma determinada chave depende da sequência de sondagem $hash(key, i)$ fornecida pela função de espalhamento
 - Observe que existem $M!$ permutações possíveis para a sequência de sondagem.
- A sondagem linear é o método mais simples de gerar a sequência de espalhamento
 - $hash(key, i) = (hash'(key) + i) \% M$

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Embora seja o método mais fácil de implementar, a sondagem linear nos leva ao problema de **agrupamento primário**.

hash	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	...	M-1
chave	60	11	51	71	20	91	90	61	40	21	151	20	-1	-1	-1	-1	-1	-1		-1

- Observe que qualquer posição da tabela pode ser utilizada por qualquer chave
 - Uma posição h calculada pela função de espalhamento deixa de ser a única posição em que a chave buscada pode ser encontrada
 - Essa passa a ser a primeira posição mais provável de encontrar elementos cujo *hash* da chave é h .

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Embora seja o método mais fácil de implementar, a sondagem linear nos leva ao problema de **agrupamento primário**.

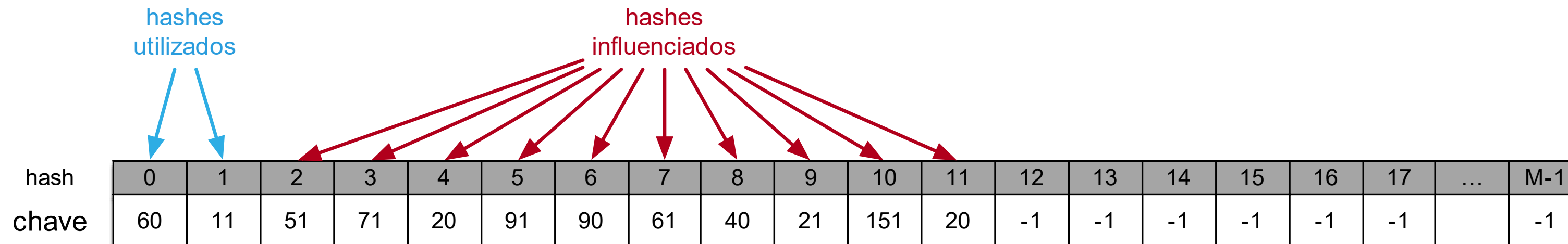
hash	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	...	M-1
chave	60	11	51	71	20	91	90	61	40	21	151	20	-1	-1	-1	-1	-1	-1		-1

- Essa característica implica que chaves mapeadas em um mesmo hash não residam em um espaço independente (como na abordagem baseada em lista encadeada)
 - Ou seja, o número de chaves armazenadas com um determinado hash influencia as demais chaves armazenadas.

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Agrupamentos podem produzir longas sequências influenciando no tempo de busca das chaves mapeadas no intervalo do mesmo.



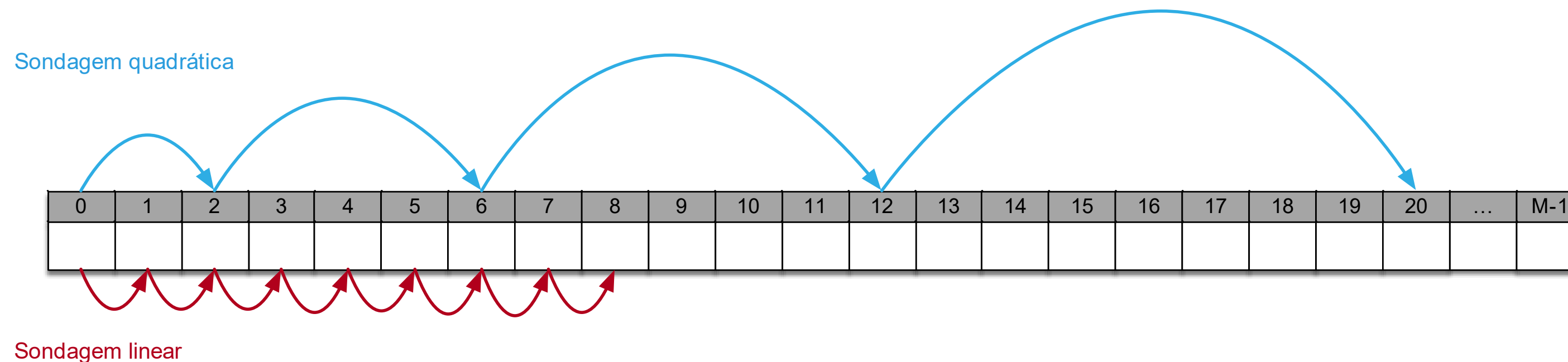
- A função de espalhamento ideal busca evitar agrupamentos ao produzir uma sequência que atenda o hash uniforme
 - Cada chave possui igual probabilidade de utilizar qualquer uma das sequências de sondagem possíveis.

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Um segundo método comum é a **sondagem quadrática**:

- $hash(key, i) = (hash'(key) + c_1i + c_2i^2) \% M$



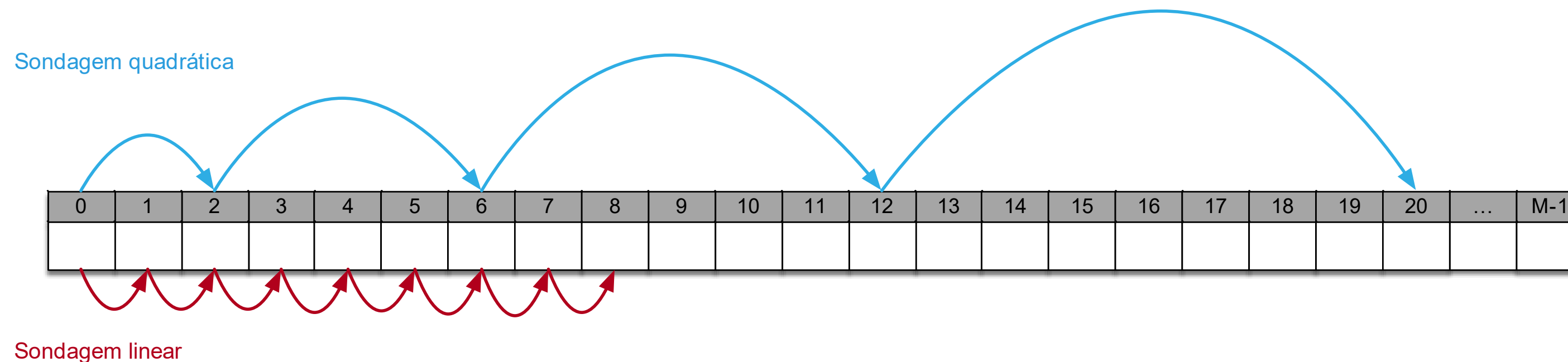
- Assim como a sondagem linear a primeira posição determina toda a sequência
 - Observe que esse método gera somente M sequências distintas dentre as $M!$ permutações possíveis.

Tabelas de dispersão

Tabela hash com endereçamento aberto

- Um segundo método comum é a **sondagem quadrática**:

- $hash(key, i) = (hash'(key) + c_1i + c_2i^2) \% M$

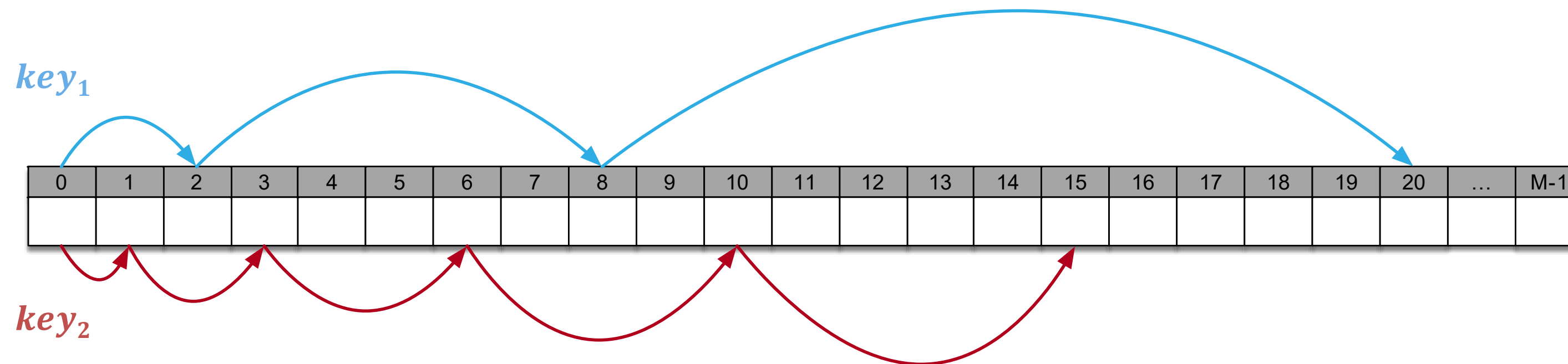


- Embora esse método apresente melhores resultados de espalhamento ele produz agrupamentos secundários
 - Todas as chaves mapeadas no mesmo hash irão ser avaliadas nas mesmas posições da sequência.

Tabelas de dispersão

Tabela hash com endereçamento aberto

- O método **hash duplo** é definido pela função a seguir:
 - $hash(key, i) = (hash_1(key) + i \cdot hash_2(key)) \% M$

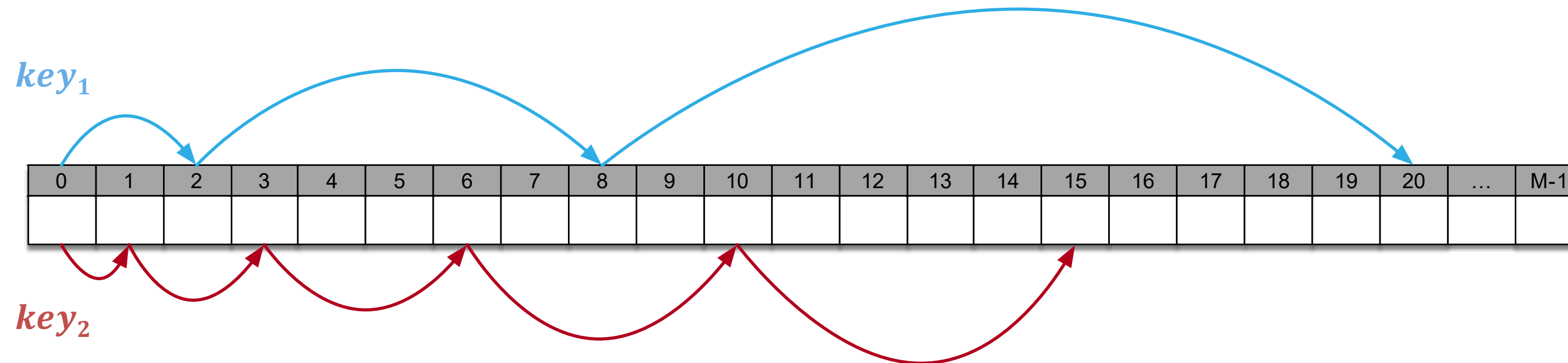


- Esse é considerado um dos melhores métodos disponíveis por produzir sequências distintas mesmo para chaves mapeadas no mesmo hash
 - Embora a posição inicial seja a mesma o deslocamento poderá ser diferente.

Tabelas de dispersão

Tabela hash com endereçamento aberto

- O método **hash duplo** é definido pela função a seguir:
 - $hash(key, i) = (hash_1(key) + i \cdot hash_2(key)) \% M$



- Essa abordagem apresenta um espalhamento melhor por ser capaz de gerar M^2 sequências distintas.

- O número de sondagens (no caso médio) para inserir uma chave em uma tabela hash de endereçamento aberto é definido por:

$$\sum_{i=0}^{\infty} \alpha^i = \frac{1}{1 - \alpha} = O(1)$$

- Exemplo: supondo que a tabela seja redimensionada quando $\alpha \geq 0.75$ temos que nesse estado:

$$1/(1 - \alpha) = 1/(1 - 0.75) = 4$$

- Observações sobre a **implementação**:
 - Requer menos espaço em memória quando comparada com a abordagem baseada em listas encadeadas
 - O espaço consumido por referências (ponteiros) para elementos da lista pode ser utilizado para definir um M maior.
 - Evita utilização de memória dinâmica para alocar cada nó individualmente.
 - Também pode ser submetida à operações de *Resize & Rehash*.
 - Ex: quando $\alpha \geq 0.7$.
- O endereçamento aberto é a abordagem utilizada pelo dicionário de Python
 - Inicia com $M = 8$ e redimensiona sempre que $\alpha \geq \frac{2}{3}$

Perguntas?

`thiago.p.araujo@fgv.br`